

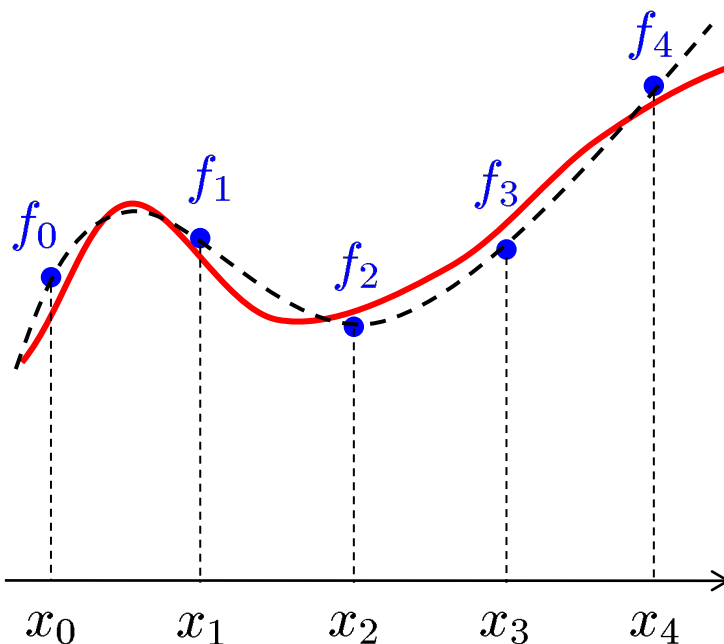


数値解析法 I 第6回講義

関数近似(1): 補間多項式・チェビシェフ近似

関数近似とは

$f(x)$: x_i での値 $f_i = f(x_i)$ のみが既知の関数 ($i = 0, 1, 2, \dots, n$)



関数近似

与えられたデータから
その**近似関数**を求めること

応用上頻繁に実行

- ✓ 回帰分析 (統計)
- ✓ 積分の近似 (数値積分)

.....

「近似関数」の求め方：**色々な方法が存在**

距離を最小にする多項式, データ点を通る多項式, ...

代表的・基本的な方法について講義

多項式近似

関数近似

与えられたデータからその近似関数を求めること

色々な種類が存在: 三角関数, 指数関数, **多項式**, ...

ところで...

多項式について考察
(理由: 扱いが容易など)

疑問: 多項式で関数近似しても大丈夫?

回答: 連続関数ならばワイエルストラスの近似定理よりOK

定理3.1 (ワイエルストラスの近似定理)

$f(x)$: $[a, b]$ 上で連続な関数 $(-\infty < a < b < \infty)$. このとき

$$\lim_{n \rightarrow \infty} \max_{a \leq x \leq b} |p_n(x) - f(x)| = 0$$

となる多項式列 $\{p_n(x)\}$ ($p_n(x)$ は n 次多項式) が存在する.

【証明】 [宮岡・永倉] 参照

近似対象が連続関数ならば多項式で近似可能

疑問: 近似多項式 $p_n(x)$ を具体的に求めるには?

補間多項式

目的: 近似多項式 $p_n(x)$ を具体的に求める

一つの方法: テイラー展開

一定精度の近似多項式を得るには, 高階微分係数値が必要



実用上の観点から

一定精度で近似可能であり, 使用するデータはせいぜい
1階微分係数値程度までで構成したい

一つ的答案: 補間多項式 (ラグランジュ補間多項式)

補間多項式とは？

- $f(x)$: $[a, b]$ 上で定義された関数
- $a \leq x_0 < x_1 < x_2 < \cdots < x_n \leq b$: 相異なる点
- 仮定：関数値 $f_i = f(x_i)$ ($i = 0, 1, 2, \dots, n$) が与えられている



定義3.1 (補間, 補間多項式)

$f(x)$ ($x \in [a, b]$, $x \neq x_i$) を求めることを**補間する (内挿する)**という。また

$$p(x_i) = f_i \quad (i = 0, 1, 2, \dots, n)$$

を満たす多項式を**補間多項式**とよぶ。

疑問: 補間多項式は一意に存在するのか？

定理3.2 (補間多項式の一意存在性)

相異なる $n + 1$ 点に対する補間条件 $p_n(x_i) = f_i$ を満たす n 次多項式 $p_n(x)$ はただ一つ存在する.

【証明概略】

$$p_n(x) = \sum_{j=0}^n a_j x^j \text{ とおくと}$$

$$\begin{aligned} p_n(x_i) &= f_i \\ (i &= 0, 1, \dots, n) \end{aligned}$$



一意可解 (行列式 $\prod_{i>j} (x_i - x_j) \neq 0$ より)

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ 1 & x_2 & x_2^2 & \cdots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_n \end{pmatrix}$$



$\{f_i\}$ に対して係数 $\{a_i\}$ は一意に定まる
($p_n(x)$ はただ一つ存在)

補間多項式の求め方

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ 1 & x_2 & x_2^2 & \cdots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_n \end{pmatrix} \Rightarrow p_n(x) = \sum_{j=0}^n a_j x^j$$

次数が増えるほどコスト（計算時間など）が増大

↓ $p_n(x)$ の表現を変更

n 次ラグランジュ (Lagrange) 補間多項式

$$p_n(x) = \sum_{i=0}^n f_i L_i^n(x) \cdot L_i^n(x_j) = \delta_{ij} = \begin{cases} 1 & (i = j) \\ 0 & (i \neq j) \end{cases}$$

n 次多項式

具体的な表現は？

$$p_n(x) = \sum_{i=0}^n f_i L_i^n(x). \quad L_i^n(x_j) = \delta_{ij} = \begin{cases} 1 & (j = i) \\ 0 & (j \neq i) \end{cases}$$

x_j ($j = 0, 1, \dots, i-1, i+1, \dots, n$): n 次多項式 $L_i^n(x)$ の解

条件 $L_i^n(x_i) = 1$ より決定

$$L_i^n(x) = c_i (x - x_0)(x - x_1) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)$$

$$L_i^n(x) = \frac{(x - x_0)(x - x_1) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_0)(x_i - x_1) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)}$$

基本多項式 (ラグランジュ基底)

例題 (ラグランジュ補間多項式)

$x_0 = 1, x_1 = 2, x_2 = 3, x_3 = 4, f_0 = 0, f_1 = 0, f_2 = 2, f_3 = 3.$

このとき 3 次ラグランジュ補間多項式 $p_3(x)$ を求めよ.

【解答】

$$L_0^3(x) = \frac{(x-2)(x-3)(x-4)}{(1-2)(1-3)(1-4)} = -\frac{1}{6}(x-2)(x-3)(x-4)$$

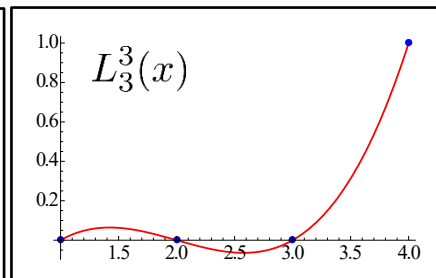
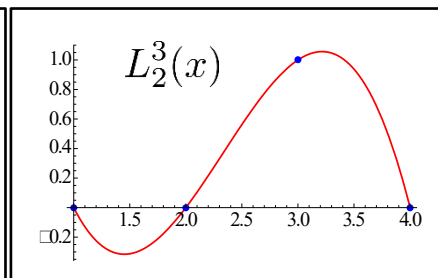
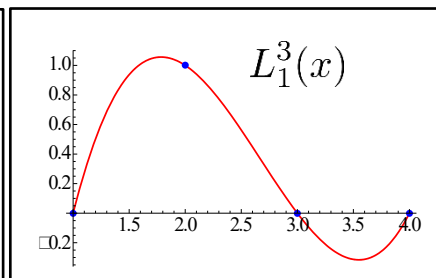
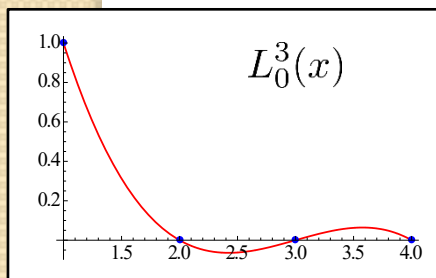
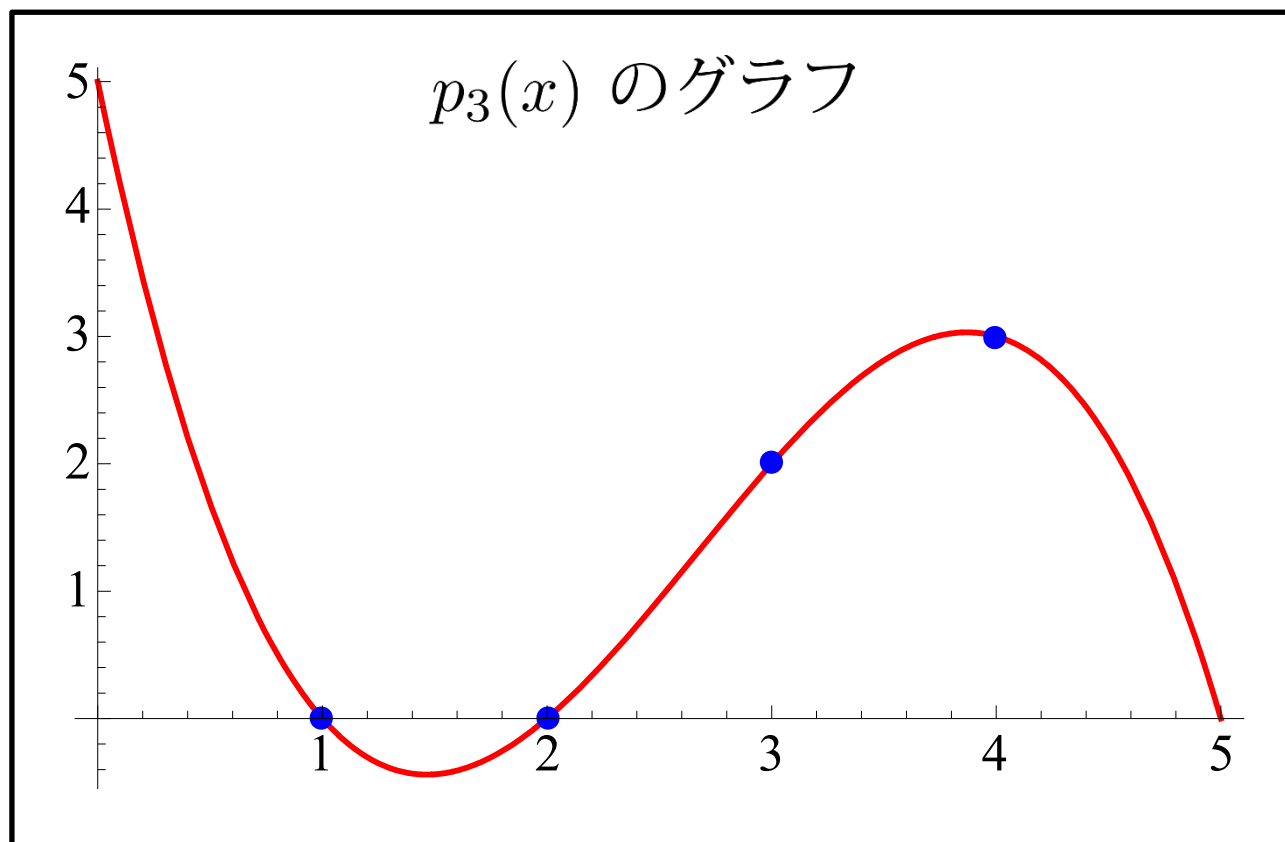
$$L_1^3(x) = \frac{(x-1)(x-3)(x-4)}{(2-1)(2-3)(2-4)} = \frac{1}{2}(x-1)(x-3)(x-4)$$

$$L_2^3(x) = \frac{(x-1)(x-2)(x-4)}{(3-1)(3-2)(3-4)} = -\frac{1}{2}(x-1)(x-2)(x-4)$$

$$L_3^3(x) = \frac{(x-1)(x-2)(x-3)}{(4-1)(4-2)(4-3)} = \frac{1}{6}(x-1)(x-2)(x-3)$$



$$p_3(x) = \sum_{i=0}^3 f_i L_i^3(x) = (x-1)(x-2) \left(-\frac{1}{2}x + \frac{5}{2} \right)$$



ニュートンの補間公式

$$p_n(x) = \sum_{i=0}^n f_i L_i^n(x).$$

ただし

$$L_i^n(x) = \frac{(x - x_0)(x - x_1) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_0)(x_i - x_1) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)}$$

点の取り方によって、**桁落ち・アンダーフロー**の危険有

別表現が必要

ニュートンの補間公式

差分商

$$f[x_0] = f(x_0), \quad f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0}, \dots,$$

$$f[x_0, x_1, \dots, x_n] = \frac{f[x_1, x_2, \dots, x_n] - f[x_0, x_1, \dots, x_{n-1}]}{x_n - x_0}$$



定理3.3 (ニュートンの補間公式)

$f(x)$ の n 次ラグランジュ補間多項式 $p_n(x)$ は、次の通りに表現できる.

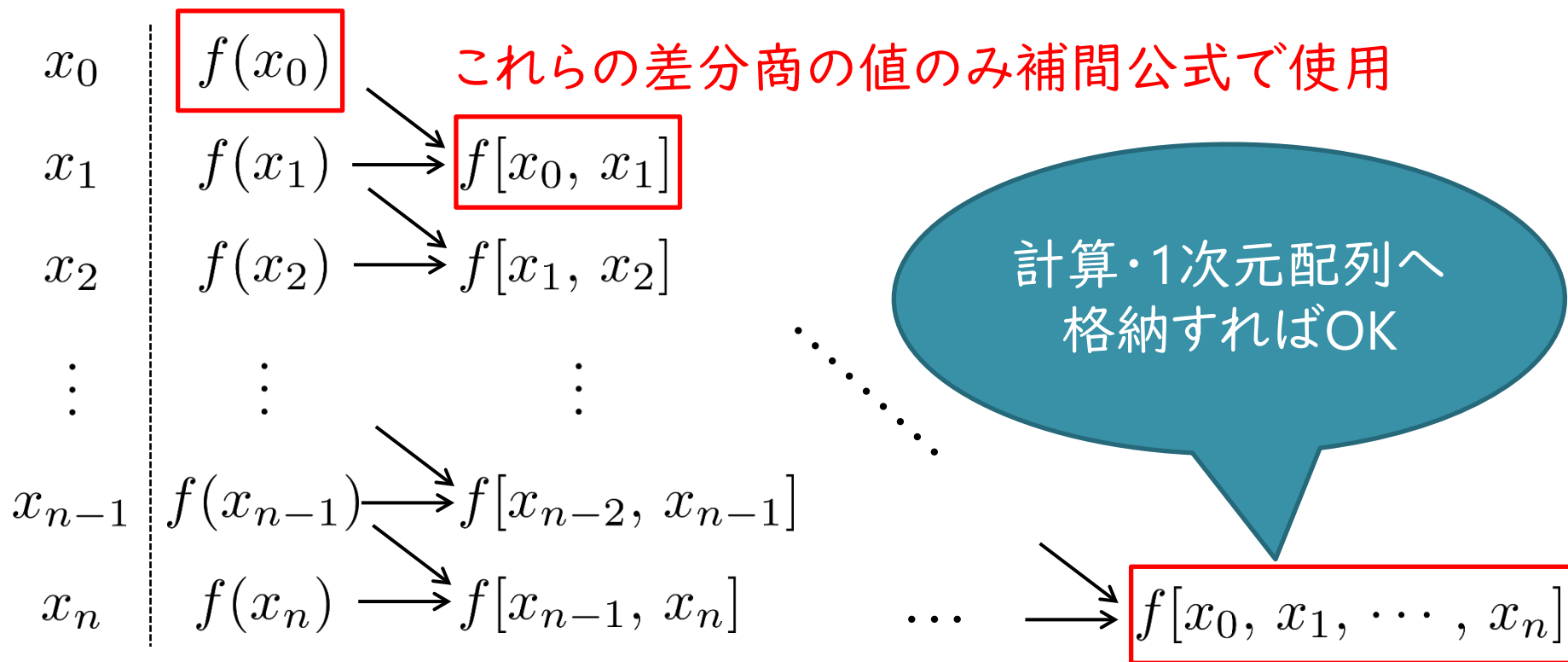
$$\begin{aligned} p_n(x) &= f[x_0] + (x - x_0)f[x_0, x_1] + \dots + \prod_{i=0}^{n-1} (x - x_i) f[x_0, x_1, \dots, x_n] \\ &= p_{n-1}(x) + \prod_{i=0}^{n-1} (x - x_i) f[x_0, x_1, \dots, x_n]. \end{aligned}$$

【証明】 第6回補足資料参照

ニュートンの補間公式

$$p_n(x) = f(x_0) + f[x_0, x_1](x - x_0) + \cdots + f[x_0, x_1, \cdots, x_n] \prod_{i=0}^{n-1} (x - x_i)$$

差分商の計算が必要



ニュートンの補間公式係数計算アルゴリズム

Require: 補間データ $(x_i, f(x_i))$ ($i = 0, 1, \dots, n$);
for $i = 0$ to n do
 $c_i = f(x_i)$;
end for
for $j = 1$ to n do
 for $i = n$ to j do
 $c_i = (c_i - c_{i-1}) / (x_i - x_{i-j})$;
 end for
end for



$$p_n(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \cdots + c_n \prod_{i=0}^{n-1} (x - x_i)$$

$$p_n(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \cdots + c_n \prod_{i=0}^{n-1} (x - x_i)$$

乗算回数 : $1 + 2 + \cdots + n = \frac{n(n+1)}{2}$



ここで...

$$p_n(x) = (((\cdots (c_n(x - x_{n-1}) + c_{n-1})(x - x_{n-2}) + c_{n-2}) \cdots + c_1)(x - x_0) + c_0$$



ホーナー
の算法

Require: 補間値を計算する点 x ;

$p = c_n$

for $i = n - 1$ to 0 **do**

$p = p \cdot (x - x_i) + c_i$;

end for

return x における補間値 p ;

乗算回数
 n 回

演習1

- ① ニュートンの補間公式により関数を補間するプログラムを作成しなさい。

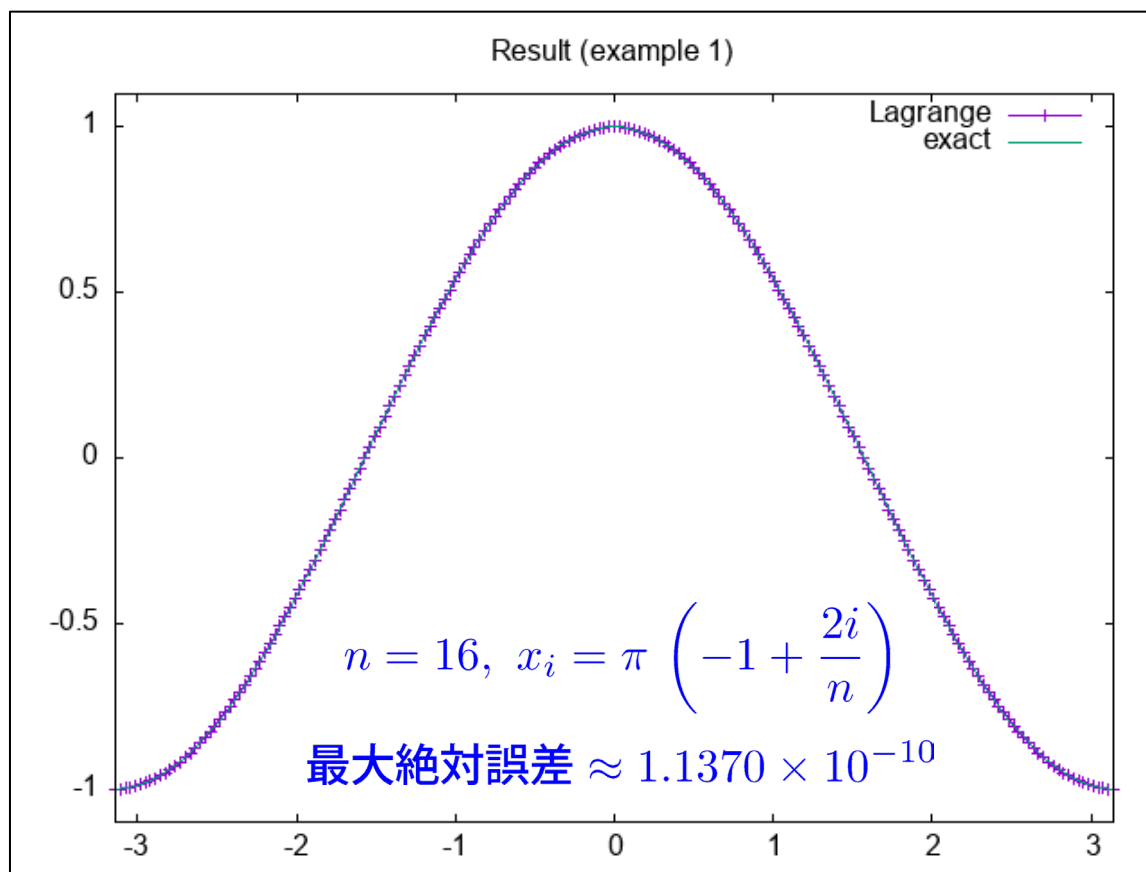
補足 (いつもの)

- プログラム言語は、**何でもOK**です。自分の最も得意な言語を使用して作成してください。
- **1から作るのは難しい**という人は、チャンネルにある**JavaまたはCプログラム(未完成)**を利用してください。

確認のための計算例1

$$f(x) = \cos x, \quad x \in [-\pi, \pi]$$

の補間多項式 $p_n(x)$ による補間値を計算し結果を図示



グラフの描画方法 (MATLAB)

- サンプルプログラム (未完成) を完成・実行する
⇒ データファイル (exc-ip.dat) が出力
- 次の手順でグラフを描画可能
 - MATLABを起動
 - “現在のフォルダー”を実行したフォルダ (ディレクトリ) へ移動
 - 次の順でコマンドを入力・Enter:
 - load exc-ip.dat
 - plot (exc_ip(:,1),exc_ip(:,2))
 - xlim([-pi pi]) ←x軸の範囲を制限したい場合に入力・Enter
 - 真の関数と一緒に描画したい場合:
 - plot(exc_ip(:,1),exc_ip(:,2),"-o",exc_ip(:,1),cos(exc_ip(:,1))))
 - xlim([-pi pi])

変数名は
ワークスペースを
確認

画像ファイル出力

Figureウィンドウの[ファイル]-[名前を付けて保存]で可能

グラフの描画方法 (gnuplot)

- 次の手順でグラフを描画可能
 - カレントディレクトリを実行したディレクトリへ移動
 - gnuplot と入力し, Enter キー
 - 次の順でコマンドを入力・Enter:
 - set xrange [-pi:pi]
 - plot 'exc-ip.dat' with lp
 - 画像ファイルを出力する場合:
 - set terminal png
 - set output 'exc-ip.png'
 - set xrange [-pi:pi]
 - plot "exc-ip.dat" with lp
 - 演習室WS (Ubuntu) で
画像ファイル閲覧: `gthumb exc-ip.png`

演習室WS
Linuxで
実行可能

フォーマット
PNG以外も有

定理3.4 (ラグランジュ補間多項式の誤差)

$f \in C^{n+1}[a, b]$, $p_n(x)$: $f(x)$ の n 次ラグランジュ補間多項式.

$\forall x \in [a, b], \exists \xi \in (\min\{x_0, x\}, \max\{x_n, x\})$;

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)(x - x_1) \cdots (x - x_n).$$

【証明】 第6回補足資料参照

$x \in [x_0, x_1]$ & $h_n = (b - a)/n$ として評価 (他の区間でも本質的には同じ)

➡ $x = x_0 + \eta h_n$ ($\eta \in [0, 1]$), $x_j = x_0 + j h_n$ ($j = 0, 1, \dots, n$)

➡ $\left| \frac{(x - x_0)(x - x_1)(x - x_2) \cdots (x - x_n)}{(n+1)!} \right|$

$$= \left| \frac{\eta \cdot (\eta - 1) \cdot (\eta - 2) \cdots (\eta - n) \cdot h_n^{n+1}}{1 \cdot 2 \cdots n \cdot (n+1)} \right|$$

$$= \left| \eta \cdot \left(\frac{\eta}{1} - 1 \right) \cdot \left(\frac{\eta}{2} - 1 \right) \cdots \left(\frac{\eta}{n} - 1 \right) \cdot \frac{h_n^{n+1}}{n+1} \right| \leq \frac{h_n^{n+1}}{n+1}$$

$$\left| \frac{\eta}{j} - 1 \right| \leq 1$$

($j = 1, 2, \dots, n$)

ラグランジュ補間多項式の誤差

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)(x - x_1) \cdots (x - x_n)$$



等分割 (分割幅 : h_n) の場合

$$\max_{x \in [a, b]} |f(x) - p_n(x)| \leq \frac{h_n^{n+1}}{n+1} \max_{x \in [a, b]} |f^{(n+1)}(x)|$$

$f \in C^\infty[a, b]$ かつ $\forall n \in \mathbb{N}, \max_{a \leq x \leq b} |f^{(n)}(x)| < C$ (C は定数)



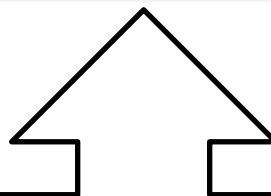
$p_n(x)$ は $f(x)$ へ収束

$$\max_{x \in [a, b]} |f(x) - p_n(x)| \leq \frac{h_n^{n+1}}{n+1} \max_{x \in [a, b]} |f^{(n+1)}(x)|$$

$f \in C^\infty[a, b]$ かつ $\forall n \in \mathbb{N}, \max_{a \leq x \leq b} |f^{(n)}(x)| < C$ (C は定数)



$p_n(x)$ は $f(x)$ へ収束

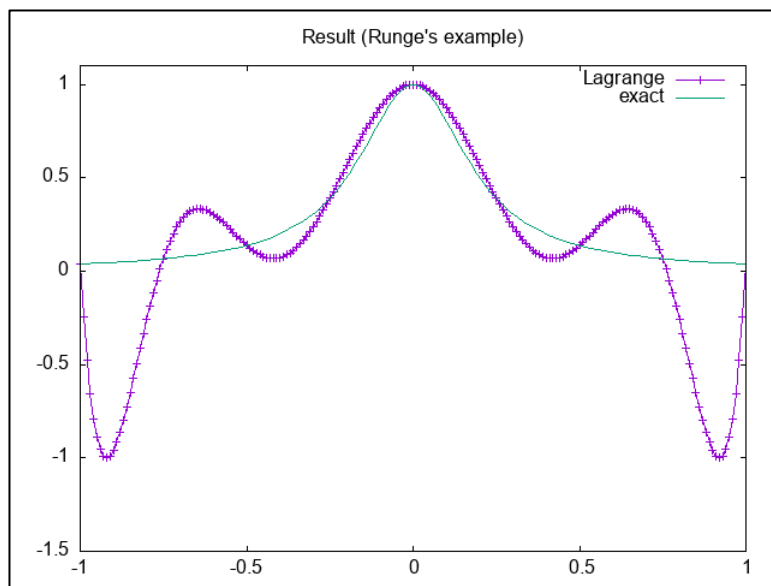


- $\sin x, \cos x$ のような関数：有効
- 微分係数値が急速に増大する関数：不適切

確認のための計算例2（ルンゲの現象）

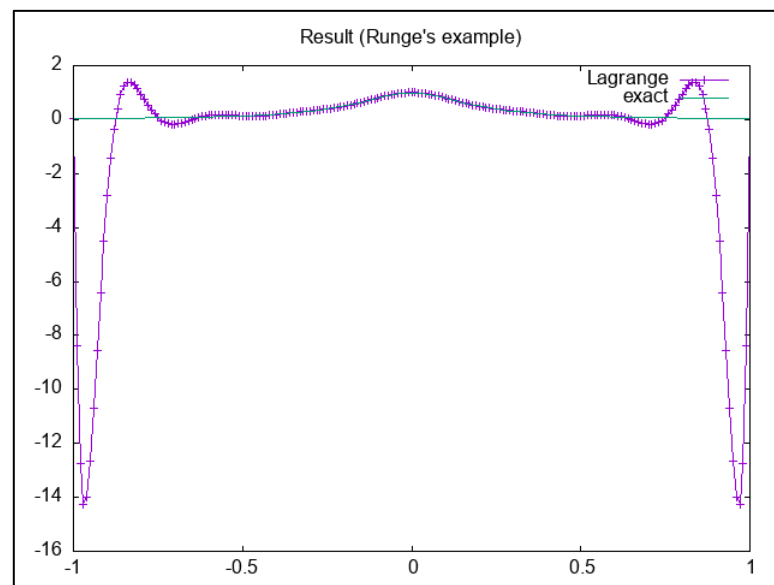
$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1]$$

$$\text{等分点 } x_i = -1 + \frac{2i}{n} \quad (i = 0, 1, \dots, n)$$



$n = 8$

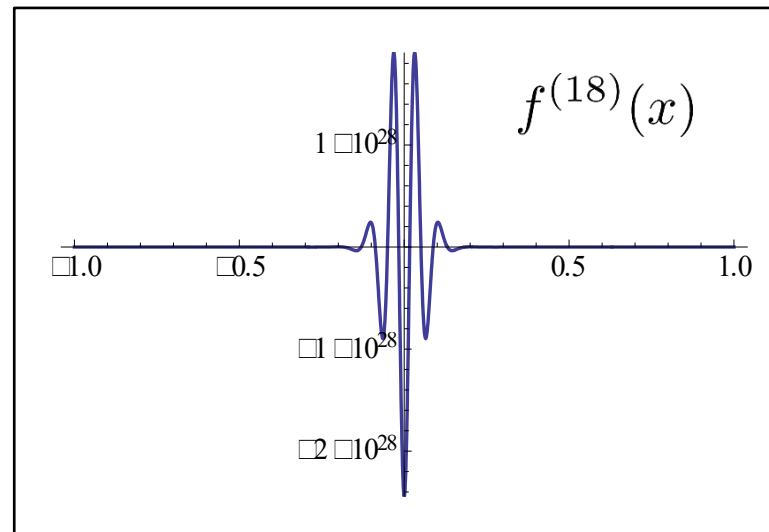
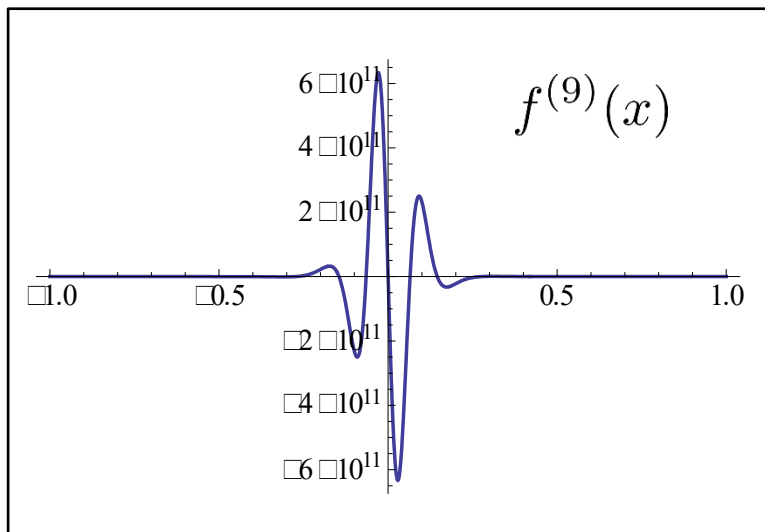
誤差 ≈ 1.0452 (約 105%)



$n = 16$

誤差 ≈ 14.321 (約 1432%)

数値解析法 | 第6回講義・関数近似(1)



$\max_{x \in [a, b]} |f^{(n)}(x)|$ は指数関数的に増大



ラグランジュ補間：この例題には不適切

他の方法は？



一つの答え：チェビシェフ近似

チェビシェフ近似

仮定：区間は $[-1, 1]$ とする

ラグランジュ補間多項式の誤差

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)(x - x_1) \cdots (x - x_n)$$

$\frac{1}{(n+1)!} \max_{x \in [-1, 1]} |(x - x_0) \cdots (x - x_n)|$ がさらに小となるよう $\{x_k\}$ を選択



誤差は小さくなる (ルンゲの現象も回避可能かも)

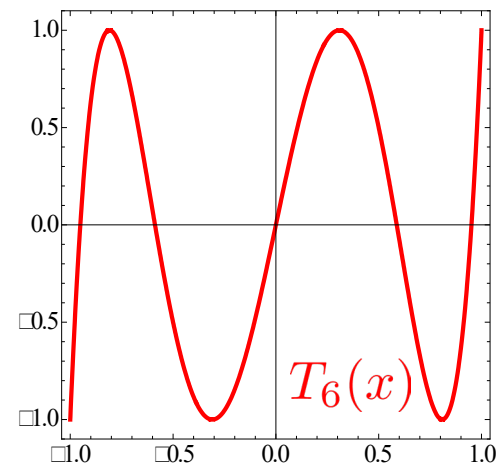
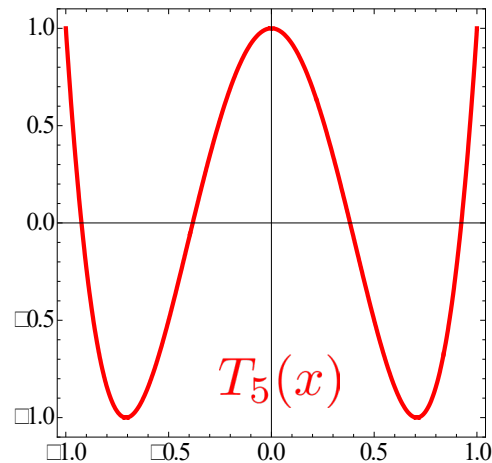
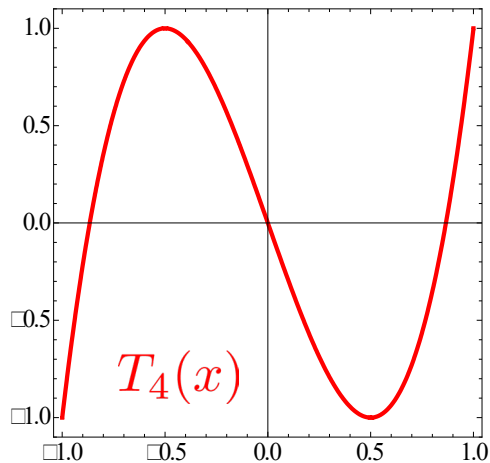
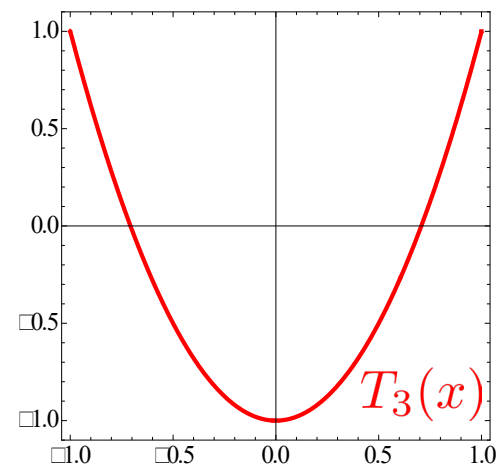
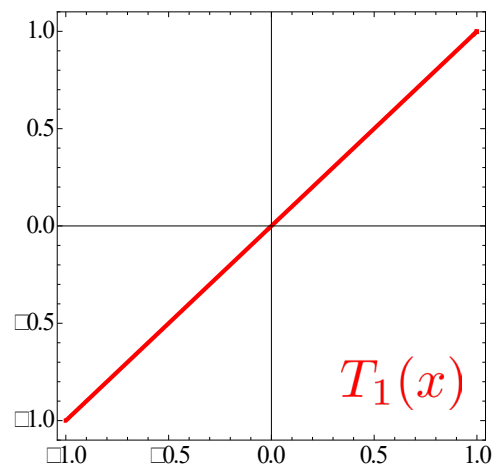
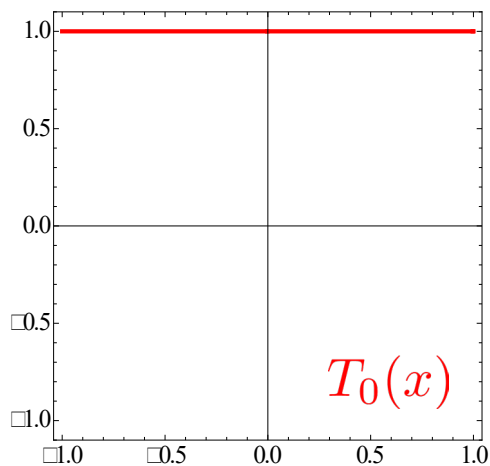
疑問：どのように選べばよいか



一つの解答：チェビシェフ多項式の零点を選べばよい

チェビシェフ多項式

$$T_n(x) = \cos(n \cos^{-1} x), \quad x \in [-1, 1], \quad n = 0, 1, 2, \dots$$



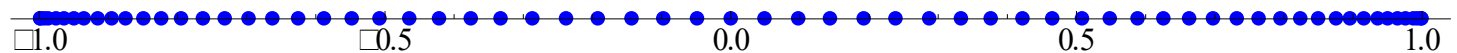
チェビシェフ多項式の性質

- $T_0(x) = 1, T_1(x) = x.$
- $T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x) \quad (n = 1, 2, 3, \dots)$

✓ 漸化式+帰納法により多項式であることを証明可能

✓ プログラム: 漸化式+再帰関数で多項式を計算可能

- $T_{n+1}(x)$ の零点: $x_k = \cos\left(\frac{\pi(k + 1/2)}{n + 1}\right) \quad (k = 0, 1, 2, \dots, n)$



$T_{64}(x)$ の零点分布

チェビシェフ多項式の性質

- 選点直交性 [杉原・室田 2]

$$\sum_{k=0}^n T_i(x_k) T_j(x_k) = \begin{cases} 0 & (i \neq j) \\ \frac{n+1}{2} & (i = j \neq 0) \\ n+1 & (i = j = 0) \end{cases} .$$

[杉原・室田2] 杉原正顯・室田一雄, 数値計算法の数理, 岩波書店, 1994.

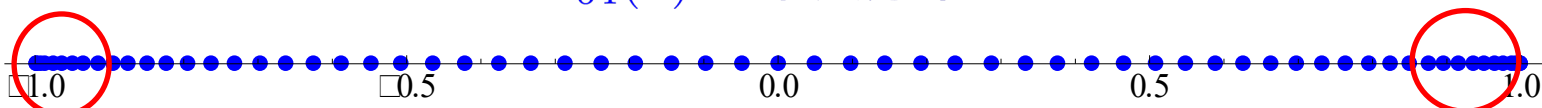
ラグランジュ基底を使用する場合の問題点

$$p_n(x) = \sum_{i=0}^n f_i L_i^n(x).$$

ただし

$$L_i^n(x) = \frac{(x - x_0)(x - x_1) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_0)(x_i - x_1) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)}$$

$T_{64}(x)$ の零点分布



密集 $\Rightarrow L_i^n(x)$ を計算する際に問題 (桁落ち等) が発生する可能性有



別表現を用いる

係数は?

$$\text{チェビシェフ近似 (補間)} : p_n(x) = \sum_{i=0}^n c_i T_i(x)$$

係数の決定方法

$$p_n(x) = \sum_{i=0}^n c_i T_i(x)$$

$x = x_k$

両辺に $T_0(x_k)$ をかける

$$p_n(x_k) T_0(x_k) = \sum_{i=0}^n c_i T_i(x_k) T_0(x_k)$$

$p_n(x_k) = f_k$, k に関して和

$$\sum_{k=0}^n f_k T_0(x_k) = \sum_{i=0}^n c_i \sum_{k=0}^n T_i(x_k) T_0(x_k)$$

$$\sum_{k=0}^n T_i(x_k) T_0(x_k) = 0 \quad (i \neq 0), \quad \sum_{k=0}^n T_0(x_k) T_0(x_k) = n+1$$

$T_0(x) = 1$

$$\sum_{k=0}^n f_k = (n+1)c_0$$

$$c_0 = \frac{1}{n+1} \sum_{k=0}^n f_k$$

他の係数も同様に
決定可能

チェビシェフ近似 (チェビシェフ補間)

$$p_n(x) = \sum_{i=0}^n c_i T_i(x).$$

ただし

$$x_k = \cos \left(\frac{\pi(k + 1/2)}{n + 1} \right) \quad (k = 0, 1, 2, \dots, n),$$

$$c_0 = \frac{1}{n + 1} \sum_{k=0}^n f_k, \quad c_i = \frac{2}{n + 1} \sum_{k=0}^n f_k T_i(x_k) \quad (i = 1, 2, \dots, n).$$

データが与えられた時点で
1回のみ計算すればOK

チェビシェフ近似の誤差評価

チェビシェフ近似の誤差

この部分を
検討

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)(x - x_1) \cdots (x - x_n)$$

x^{n+1} の係数

$\{x_i\}_{i=0}^n: T_{n+1}(x)$ の零点



$$T_{n+1}(x) = c_{n+1} \cdot (x - x_0)(x - x_1) \cdots (x - x_n)$$

$$\begin{aligned} T_{n+1}(x) &= 2x T_n(x) - T_{n-1}(x) = 2^2 x^2 T_{n-1} - T_{n-1}(x) - 2x T_{n-2}(x) \\ &= \cdots = 2^n x^n T_1(x) + (n-1 \text{ 次以下の多項式}) \\ &= 2^n x^{n+1} + (n-1 \text{ 次以下の多項式}) \end{aligned}$$



$$(x - x_0)(x - x_1) \cdots (x - x_n) = \frac{1}{2^n} T_{n+1}(x)$$

定理3.5 (チェビシェフ近似の誤差)

$f \in C^{n+1}[-1, 1]$, $p_n(x)$: $f(x)$ の n 次チェビシェフ近似.

$\forall x \in [-1, 1], \exists \xi \in (\min\{x_0, x\}, \max\{x_n, x\})$;

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{2^n(n+1)!} T_{n+1}(x).$$

【証明】 詳しくは第6回補足資料参照


$$\max_{x \in [-1, 1]} |T_{n+1}(x)| \leq 1$$

$$\max_{x \in [-1, 1]} |f(x) - p_n(x)| \leq \frac{1}{2^n(n+1)!} \max_{x \in [-1, 1]} |f^{(n+1)}(x)|$$

$f \in C^\infty[-1, 1]$ かつ $\forall n \in \mathbb{N}, \max_{x \in [-1, 1]} |f^{(n)}(x)| < C$ (C は定数)



$p_n(x)$: 多項式オーダーより早く収束 (高精度)

一般区間のチェビシェフ近似

変数変換すればよい

$$p_n(x) = \sum_{i=0}^n c_i T_i \left(\frac{2x - (a+b)}{b-a} \right), \quad x \in [a, b].$$

ただし

$$x_k = \cos \left(\frac{\pi(k+1/2)}{n+1} \right), \quad \hat{x}_k = \frac{(b-a)x_k + a+b}{2} \quad (k = 0, 1, 2, \dots, n),$$

$$c_0 = \frac{1}{n+1} \sum_{k=0}^n f_k, \quad c_i = \frac{2}{n+1} \sum_{k=0}^n f_k T_i(x_k) \quad (i = 1, 2, \dots, n).$$



$$\max_{x \in [a, b]} |f(x) - p_n(x)| \leq \frac{2(b-a)^{n+1}}{4^{n+1}(n+1)!} \max_{x \in [a, b]} |f^{(n+1)}(x)|$$

演習2

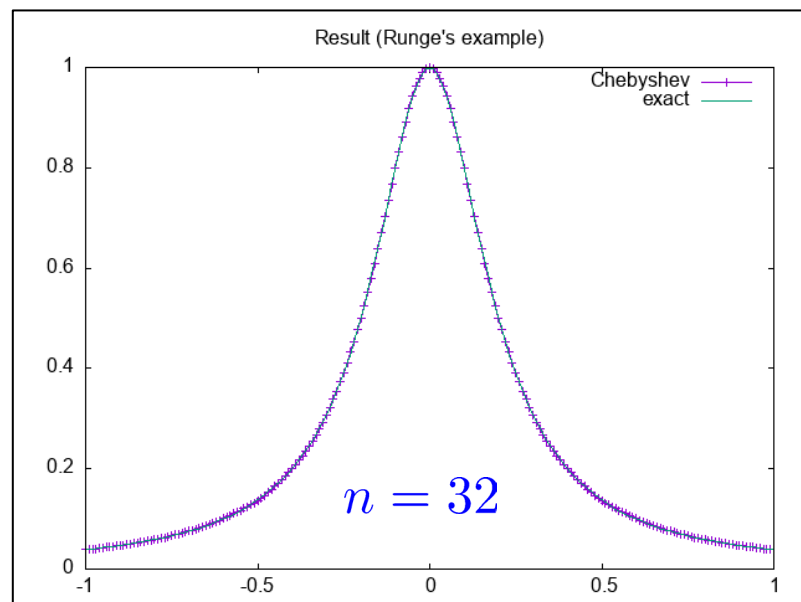
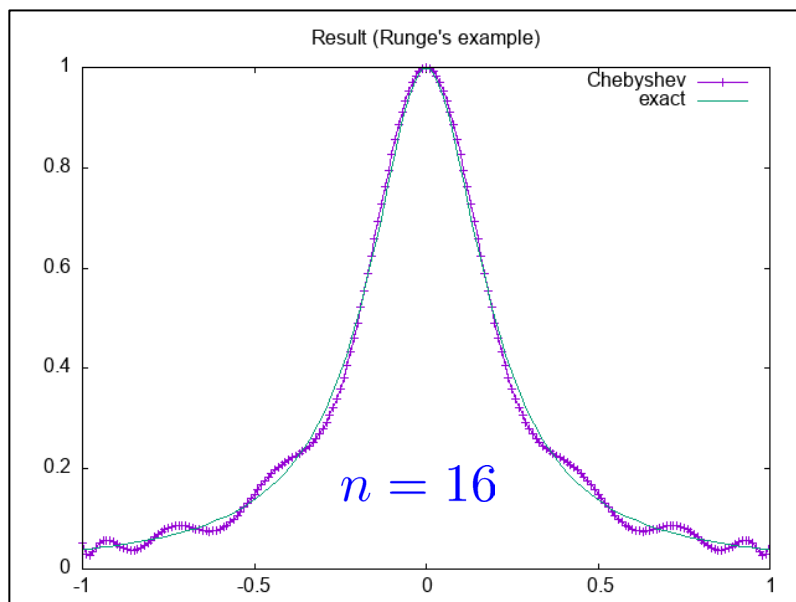
- ① チェビシェフ近似により関数を補間するプログラムを作成しなさい。

補足 (いつもの)

- プログラム言語は、**何でもOK**です。自分の最も得意な言語を使用して作成してください。
- **1から作るのは難しい**という人は、チャンネルにある**JavaまたはCプログラム (未完成)**を利用してください。

確認のための計算例3

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1]$$



ルンゲの現象は発生しない

チェビシェフ近似の問題点

$$p_n(x) = \sum_{i=0}^n c_i T_i(x).$$

分点選択
自由無し

ただし

$$x_k = \cos \left(\frac{\pi(k + 1/2)}{n + 1} \right) \quad (k = 0, 1, 2, \dots, n),$$

$$c_0 = \frac{1}{n + 1} \sum_{k=0}^n f_k, \quad c_i = \frac{2}{n + 1} \sum_{k=0}^n f_k T_i(x_k) \quad (i = 1, 2, \dots, n).$$

分点上で関数値が与えられていない: **使用不能**

他の方法は?



スプライン補間(次回講義)

まとめ（第6回講義）

- 講義内容：関数近似

- テイラー展開法

- 1点の情報のみで関数近似可能
 - 一定精度を得るには高階微分が必要（実用に不向き）

- 補間多項式（ラグランジュ）

- 基本多項式を用いた表現
 - ニュートンの補間公式
 - 近似対象の関数の性質によっては収束しない（ルンゲの現象）

- チェビシェフ近似

- チェビシェフ多項式+零点を使用
 - 利点：高精度, 欠点：分点に自由度無し

- 次回講義

- スプライン補間, 最小二乗近似

補足：ルンゲの例題の描画方法（MATLAB）

- 次の手順でグラフを描画可能
 - MATLABを起動
 - “現在のフォルダー”を実行したフォルダ（ディレクトリ）へ移動
 - 次の順でコマンドを入力・Enter：
 - `load exp-ip.dat`
 - `plot(exc_ip(:,1),exc_ip(:,2))`
 - `xlim([-pi pi])` ←x軸の範囲を制限したい場合に入力・Enter
 - 真の関数と一緒に描画したい場合：
 - `plot(exc_ip(:,1),exc_ip(:,2),'-o',exc_ip(:,1), 1 ./ (1+25*exc_ip(:,1).*exc_ip(:,1)))`
 - `xlim([-pi pi])`

補足：ライブラリ使用（GSL）

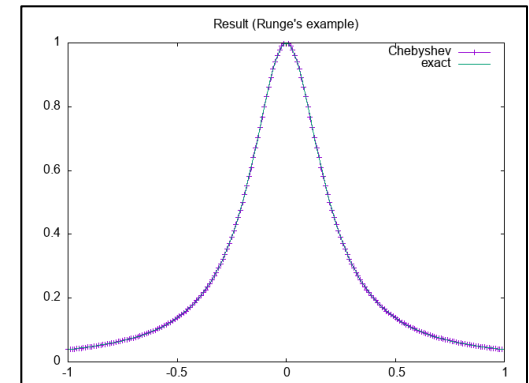
- 補間用関数

- `gsl_interp_alloc` + `gsl_interp_init` + `gsl_interp_eval`
- 多項式だけではなく他の方法も指定可能（次回）
- 使用法：`testlag_gsl.c` 参照

- チェビシェフ近似用関数

- `gsl_cheb_alloc` + `gsl_cheb_init` + `gsl_cheb_eval`
- 演習問題プログラムより高速
- 使用法：`testche_gsl.c` 参照

- 使用方法：**特殊なので注意**



$n = 64$

補足：Mathematica（補間多項式）

- **InterpolatingPolynomial**関数を使用
- 使用法：**lagip.nb** 参照

